

Módulo 06: Nivel Experto

Programación Funcional, DI, Build Runner, Dart 3 y Debugging

Guía de Referencia Rápida — Flutter Clean Architecture with Supabase

PARTE I — Programación Funcional

01. fpdart: Result Pattern con Either y Option

Objetivo: Eliminar exceptions del dominio usando tipos funcionales.

Dart tradicional	→	fpdart
try-catch / null	→	Either<L, R> / Option<A>
Excepciones ocultas	→	Errores explícitos en tipo

Conceptos Clave

- `Either<L, R>` : izquierda (error) | derecha (éxito)
- `Option<A>` : `Some(value)` | `None` — reemplaza null
- `TaskEither<L, A>` : Either diferido (async)
- `flatMap` : encadenar operaciones que retornan Either
- `fold` : convertir Either a valor final en Cubit

Puntos Clave:

- Repository retorna `Either<Failure, T>`
- UseCase propagar con `flatMap`
- Cubit consumir con `fold` : left → error state, right → success state
- `TaskEither` para operaciones async antes de ejecutar

Tip: `TaskEither.sequence()` ejecuta múltiples tareas en paralelo y retorna `Either<L, List<R>>`.

PARTE II — Dependency Injection y Arquitectura

02. DI Automatizada con Injectable + GetIt

Objetivo: Inyección de dependencias con code generation.

Flujo Injectable

`@Injectable()` → `build_runner` → `injection.config.dart` → `GetIt`

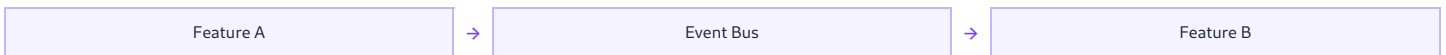
Anotación	Alcance	Uso típico
<code>@Injectable()</code>	Instancia nueva cada vez	UseCases, Services
<code>@singleton</code>	Una sola instancia	API clients, DB
<code>@lazySingleton</code>	Lazy + una instancia	Config, prefs
<code>@injectable</code>	Factory parametrizado	Parámetros dinámicos

Reglas de Oro:

- Un `@module` por librería externa (SharedPreferences, etc.)
- Un `@Environment('dev')` para cada configuración
- Usar `get_it` + `injectable` en combo
- Tests: `GetIt.instance.reset()` + `registerFactory`

03. Comunicación entre Features con Event Bus

Objetivo: Desacoplar features sin dependencias directas.



Patrones de Comunicación

- **Event Bus:** `StreamController<Event>` central
- **Service Locator:** `GetIt` como intermediario
- **Callback injection:** Funciones pasadas como parámetros
- **Reactive:** Streams combinados con `combineLatest`

Tip: Usar `BLoCListener` para reaccionar a eventos de otros features sin importar el BLoC directamente.

04. Patrones Reactivos con Streams y StreamUseCase

Objetivo: Datos en tiempo real con patrones de backpressure.

Operadores Esenciales

Operador	Función
map	Transforma cada evento
where	Filtra eventos
debounce	Ignora eventos rápidos
throttle	Limita frecuencia
sample	Toma el último valor en intervalo
switchMap	Cambia a nuevo stream
combineLatest	Combina múltiples streams

Arquitectura:

- DataSource → Stream (Supabase Realtime, WebSocket)
- Repository → Stream transformado
- StreamUseCase → Lógica de negocio reactiva
- Cubit → StreamSubscription con cancel() en close()

PARTE III — Build Runner y Code Generation

05. Build Runner: Comandos y Arquitectura

Objetivo: Dominar el ecosistema de generación de código.

Comando	Uso
dart run build_runner build	Build completo
--delete-conflicting-outputs	Eliminar conflictos (siempre usar)
watch	Modo observación para desarrollo
clean	Limpiar archivos generados

Arquitectura Build Runner
Builder → BuilderDefinition → build.yaml → source_gen
Cada paquete registra Builders que transforman archivos fuente en archivos generados.

Mejores Prácticas:

- Los .g.dart se comitan en el repo
- analysis_options.yaml debe excluir archivos generados
- Usar Makefile o scripts para estandarizar comandos

06. json_serializable y Freezed

Objetivo: Data classes inmutables con serialización automática.

Clase anotada → build_runner → .g.dart + .freezed.dart

Combinación Ideal

- @freezed : inmutabilidad, copyWith, ==, hashCode
- @JsonSerializable() : fromJson / toJson automáticos
- Freezed genera la estructura, json_serializable la serialización

Regla: Si un modelo tiene 3+ campos, usa freezed + json_serializable. El tiempo ahorrado justifica la configuración inicial.

07. Retrofit: Clientes HTTP Generados

Objetivo: APIs tipadas con interceptors y generación automática.

Pilares de Retrofit

- @RestApi() : define la interfaz HTTP
- @GET/@POST/@PUT/@DELETE : endpoints anotados
- Dio: motor HTTP subyacente
- Interceptors: auth, logging, retry, errores

Integración:

- Retrofit + json_serializable: modelos generados automáticamente
- Retrofit + injectable: cliente registrado en DI
- Retrofit + freezed: respuestas inmutables con patrón Either

08. Flutter Gen: Assets, Splash, Iconos y Localización

Objetivo: Gestión segura de assets e internacionalización.

Herramientas del Ecosistema

- flutter_native_splash: splash screen nativa por plataforma
- flutter_launcher_icons: iconos de app generados
- flutter_gen: assets tipados con GenAssets
- l10n: localización con ARB files

Tip: Activar generate: true en pubspec.yaml habilita todo el ecosistema de code generation para assets y localización.

PARTE IV — Dart 3 y Debugging

09. Dart 3 Pattern Matching Nativo

Objetivo: Manejo exhaustivo de tipos sin dependencias externas.

Sintaxis Dart 3

```
switch (valor) {
  int n when n > 0 => 'Positivo',
  0               => 'Cero',
  int n           => 'Negativo',
  String s        => 'Texto: $s',
  -               => 'Otro tipo',
}
```

Características:

- Sealed classes: jerarquías de tipos con casos exhaustivos
- Guards: when para condiciones adicionales
- Destructuring: extraer campos de objetos y listas
- Alternatives: || para combinar patrones
- Sin dependencias externas — compilador fuerza exhaustividad

vs fpdart: Dart 3 pattern matching es ideal para estados sealed. fpdart (Either/Option) es mejor para flujos funcionales encadenados.

10. Debugging build_runner y Code Generation

Objetivo: Diagnosticar y resolver problemas comunes de generación.

Errores Frecuentes

Error	Causa	Solución
Conflicting outputs	Archivos .g.dart huérfanos	--delete-conflicting-outputs
Could not find	import no resuelto	flutter pub get
Cyclic dependency	imports circulares	Reorganizar imports
Build failed	Error en anotación	Revisar sintaxis de anotaciones

Checklist de Debugging:

- flutter pub get ejecutado?
- dart analyze sin errores?
- ¿Hay imports circulares?
- ¿Se usa --delete-conflicting-outputs ?
- ¿Se limpió con clean antes?

Comando de emergencia: dart run build_runner clean && dart run build_runner build --delete-conflicting-outputs